Mammon V3 PRD - Part 3: Python Cognitive Pipeline (Agent Q)

1. Semantic Dimensionality Reduction

Target: cognitive_node/semantic_watcher.py

Raw mathematical floats (e.g., Variance = 0.000000004) confuse LLMs. The Python pipeline must convert raw Redis telemetry into institutional-grade semantic state vectors before feeding it to the AI.

```
def generate_state_vector(symbol: str) -> str:
    # Fetch 5m arrays from Redis/Postgres
    # 1. Micro-Volatility: StdDev of 1s returns * 10000 -> Basis Points (BPS)
    # 2. TFI Z-Score: (Current TFI - Mean TFI) / StdDev TFI
    # 3. Adverse Selection: % of trades in last 15m where PnL was negative at T+1s

    return f"""
    [ORACLE 5M STATE VECTOR - {symbol}]
    1. Volatility: {vol_bps:.2f} bps/sec ({'EXTREME' if vol_bps > 5 else 'NORMAL'})
    2. Toxicity: {tfi_zscore:+.2f}σ ({'TOXIC' if abs(tfi_zscore) > 2.0 else 'CLEAN'})
    3. Adverse Selection: {adv_sel_pct}% ({'DANGER' if adv_sel_pct > 60 else 'SAFE'})
    """
```

2. The Reinforcement Learning (RL) Reward

Target: cognitive_node/memory_ledger.py

The system evaluates the previous 3-minute tactical cycle using a ruthless Cadence-First reward function.

```
def calculate_reward(trips: int, win_rate: float, pnl: float) -> float:
    # Target: 15 Trips per 3 minutes
    if trips < 15:
        # Severe exponential penalty for failing volume quota
        volume_score = -200.0 * (1.0 - (trips / 15.0))
    else:
        volume_score = 50.0 + (trips - 15.0) * 2.0

    win_score = (win_rate - 0.5) * 50.0
    pnl_score = pnl * 10.0
```

```
    return volume_score + win_score + pnl_score
```

3. Agent Q Hierarchy & Prompts

Target: cognitive_node/agent_q_loop.py and prompts/

The 5m Oracle (Regime Classifier)

Reads the Semantic State Vector and classifies the macro environment.

- **Output:** {"regime": "RETAIL_FRENZY_UP"} (or MEAN_REVERTING, TOXIC_CASCADE, INSTITUTIONAL_DUMP).

The 3m Tactical Alpha (The Sniper)

Receives the Oracle's Regime, the State Vector, and its own RAG Memory (T-1 Reward).

- **Output Schema:** {"momentum_trigger_obi": float, "take_profit_ticks": int, "stop_loss_ticks": int, "max_active_tranches": int}
- **Instruction:** "If your T-1 Reward is negative due to < 15 trades, you MUST lower momentum_trigger_obi to 0.40 to trigger attacks more frequently."

The 3m Risk Manager (The Bounds)

Receives the Alpha's proposal and forcefully clamps it based on the Oracle Regime.

- **Instruction:** "If Regime is TOXIC_CASCADE, you MUST override momentum_trigger_obi to ≥ 0.90 and set max_active_tranches to exactly 1. Do not let the bot catch falling knives."

4. Orchestration Loop

The agent_q_loop.py script runs continuously:

1. Every 5 minutes: Fetch State Vector $\rightarrow$ Run Oracle $\rightarrow$ Publish Regime to Redis.
2. Every 3 minutes: Fetch State Vector & RAG Memory $\rightarrow$ Run Alpha $\rightarrow$ Run Risk $\rightarrow$ Publish Sniper Levers to Redis hft:live_params:{symbol}.